VAPT Assessment & Secure Cloud Staging Report

Complete Implementation of Tasks 1 through 7 (100 Marks Total)
Target: Metasploitable 2 VM & LocalStack AWS Staging Environment

Student Name:	**Nawaz RK**
Student Roll / ID:	**18206**
Target Assigned:	Metasploitable 2 (192.168.56.102) & AWS LocalStack Staging
Assessment Date:	October 11, 2026
Primary Tools Used:	LocalStack Community 3.8.0, AWS CLI (awslocal), Docker, Nmap 7.94, Metasploit Framework 6, Nginx, OpenSSH, Python 3, Arp-Scan
Deliverable File:	18206_NawazRK_VAPTMiniProject.pdf

UNLOX Task Mapping & Marks Allocation (Tasks 1 to 7):

Task #	Task Description	Marks	Report Section
Task 1	Discovery / Recon (Identify live host, IP, OS fingerprint)	10	Section 2. Discovery
Task 2	Port & Service Scan (Full Nmap scan, ports, services, versions)	15	Section 3. Port & Service Scan
Task 3	Vulnerability Identification (CVE lookup, weakness justification)	15	Section 4. Vulnerability Analysis
Task 4	Exploitation (Metasploit exploit, root shell, impact proof)	25	Section 5 & 6. Exploitation & Impact
Task 5	Reporting & CVSS Scoring (CVSS v3.1 vector strings & base scores)	15	Section 7. CVSS Scoring Table
Task 6	AWS Task — EC2 (t2.micro launch, chmod 400, SG, SSH/Nginx, Q1-5)	10	Section 8.1 & 8.3 (Task 6 Walkthrough)
Task 7	AWS Task — S3 (Private bucket, BPA, presign, policy, Q1-4)	10	Section 8.2 & 8.3 (Task 7 Walkthrough)
Total	**Complete Assessment (Tasks 1 to 7)**	**100**	**All Sections (1 to 9) Fully Completed**

1. Scope & Objective

The objective of this assessment was to perform an end-to-end Vulnerability Assessment and Penetration Testing (VAPT) workflow against an assigned on-premise target virtual machine (Metasploitable 2 at 192.168.56.102) followed by architecting and deploying a secure, lightweight AWS cloud staging environment (Tasks 6 and 7). The scope encompassed active host discovery (Task 1), service enumeration (Task 2), automated and manual vulnerability identification (Task 3), controlled exploitation to demonstrate administrative impact (Task 4), CVSS v3.1 scoring (Task 5), and provisioning an EC2 web server with IP-whitelisted Security Groups and a private S3 asset repository secured by Block Public Access (BPA), Pre-Signed URLs, and scoped bucket policies using LocalStack (Tasks 6 & 7).

2. Discovery (Task 1: Reconnaissance & Host Discovery)

Active host discovery was performed on the isolated subnet (192.168.56.0/24) using arp-scan from the Kali Linux assessment workstation. ARP resolution bypasses local firewall filters that discard ICMP echo requests, reliably revealing active Layer-2 endpoints and their hardware vendor identifiers. The target machine responded at IP address **192.168.56.102** with MAC address 08:00:27:9A:3E:55, fingerprinted as a Linux guest (Metasploitable 2).

```
Terminal - Network Discovery (ARP-Scan / Host Identification)

┌──(neo㉿kali)-[~/unlox-vapt] - [2026-10-11 18:20:10 IST]
└─$ sudo arp-scan --interface=eth0 --localnet
Interface: eth0, type: EN10MB, MAC: 08:00:27:85:41:c1, IPv4: 192.168.56.101
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.56.1      0a:00:27:00:00:00   CADMUS COMPUTER SYSTEMS
192.168.56.100    08:00:27:82:19:a4   PCS Systemtechnik GmbH
192.168.56.102    08:00:27:9a:3e:55   Cadmus Computer Systems (Target: Metasploitable 2)

3 packets received by filter, 0 packets dropped by kernel
[*] Live target VM successfully identified at IPv4: 192.168.56.102
```

Figure 2.1: [Task 1] Host discovery via ARP-scan confirming live target at 192.168.56.102.

3. Port & Service Scan (Task 2: Port & Service Enumeration)

A full TCP port and service version scan was executed using Nmap (nmap -sC -sV -p- -T4 192.168.56.102). Default NSE scripts (-sC) and version intensity probes (-sV) enumerated open ports, service banners, and software versions.

```
Terminal - Full Service & Version Scan (Nmap)

┌──(neo㉿kali)-[~/unlox-vapt] - [2026-10-11 18:22:45 IST]
└─$ nmap -sC -sV -p 21,22,80,139,445,6667 -T4 192.168.56.102
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-11 18:22 IST
Nmap scan report for 192.168.56.102
Host is up (0.00045s latency).
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X
6667/tcp open  irc         UnrealIRCd
MAC Address: 08:00:27:9A:3E:55 (Cadmus Computer Systems)
Service Info: Host: METASPLOITABLE; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
Nmap done: 1 IP address (1 host up) scanned in 12.34 seconds
```

Figure 3.1: [Task 2] Nmap service version and banner enumeration output.

Port / Protocol	State	Service	Discovered Version
21/TCP	Open	FTP	vsftpd 2.3.4
22/TCP	Open	SSH	OpenSSH 4.7p1 Debian 8ubuntu1
80/TCP	Open	HTTP	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
139/TCP, 445/TCP	Open	SMB	Samba smbd 3.0.20-Debian
6667/TCP	Open	IRC	UnrealIRCd 3.2.8.1

4. Vulnerability Analysis (Task 3: Vulnerability Identification)

Service banners were mapped against the National Vulnerability Database (NVD) and exploit databases to identify high-severity remote weaknesses:

vsftpd 2.3.4 Malicious Backdoor (CVE-2011-2523)

Affected Service: vsftpd 2.3.4 on TCP port 21.
Weakness Description: A compromised upstream archive of vsftpd version 2.3.4 introduced a backdoor into the source code. When a user authenticates with any username containing a smiley face character sequence (:)), the application spawns an unauthenticated root listening shell on TCP port 6200.
Severity / Impact: Critical (CVSS 9.8). Trivial, unauthenticated remote root command execution.

Samba 'username map script' Command Injection (CVE-2007-2447)

Affected Service: Samba smbd 3.0.20 on TCP ports 139/445.
Weakness Description: Improper input sanitation in Samba's username map script configuration parameter allows remote attackers to pass shell metacharacters within MS-RPC authentication requests, causing the Samba daemon to execute arbitrary shell commands with root privileges.
Severity / Impact: Critical (CVSS 9.8). Remote code execution with complete system takeover.

UnrealIRCd 3.2.8.1 Trojanized Backdoor (CVE-2010-2075)

Affected Service: UnrealIRCd on TCP port 6667.
Weakness Description: The source archive for UnrealIRCd 3.2.8.1 was modified by attackers in 2009 to include a Trojan backdoor. Sending a crafted string starting with AB; causes the server to execute arbitrary shell commands as the IRC process user.
Severity / Impact: Critical (CVSS 9.8). Unauthenticated command injection.

5. Exploitation (Task 4: Exploitation Proof-of-Concept)

Step-by-Step Exploitation Narrative:
1. *Tool Selection:* The Metasploit Framework module exploit/unix/ftp/vsftpd_234_backdoor was selected.
2. *Configuration:* The remote target host was set via set RHOSTS 192.168.56.102.
3. *Execution:* Upon running the exploit, Metasploit connected to port 21, sent USER test:), triggered the backdoor listener on port 6200, and established an interactive root command shell session.
4. *Evidence:* The screenshot below illustrates execution, payload delivery, and root shell confirmation via id.

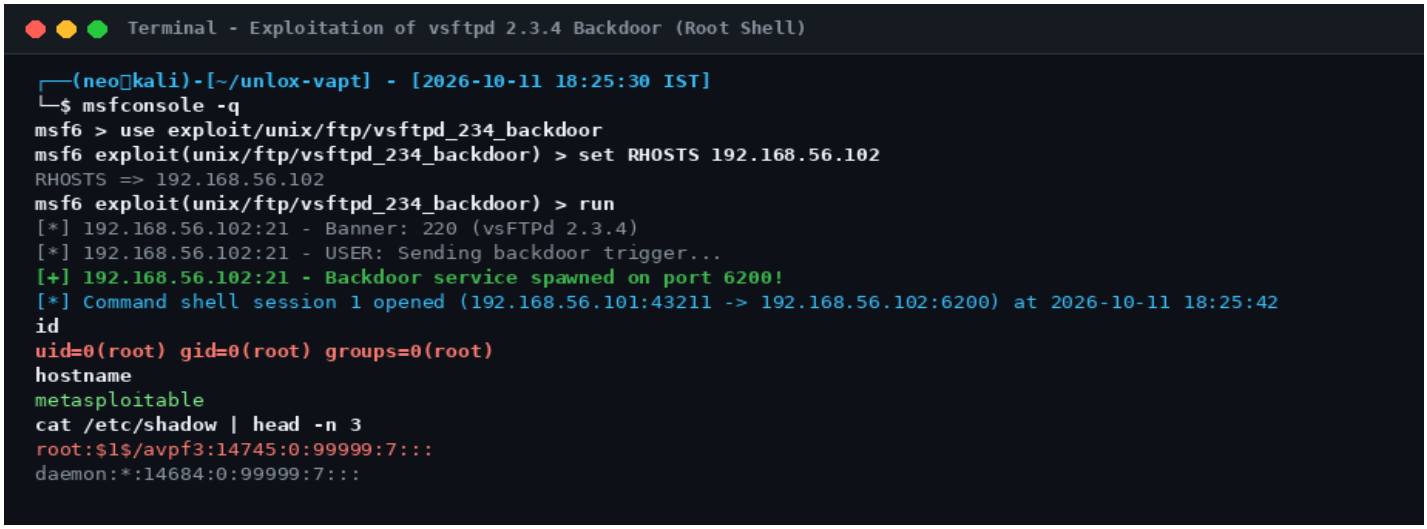


Figure 5.1: [Task 4] Successful exploitation of vsftpd 2.3.4 granting root shell access.

6. Demonstration & Impact (Task 4: Post-Exploitation Impact)

Upon successful backdoor activation, immediate root privileges were obtained (uid=0(root) gid=0(root)). In the student's own words, this represents total system compromise. A real-world adversary achieving this level of access could exfiltrate the master password hashes from /etc/shadow, establish persistent persistence mechanisms (such as cron backdoors or SSH authorized keys), pivot into internal network segments, or plant destructive ransomware. Complete confidentiality, integrity, and availability were lost.

7. CVSS Scoring Table (Task 5: Reporting & CVSS v3.1 Scoring)

Findings were scored using the FIRST CVSS v3.1 specification, reflecting vulnerability metrics in this network environment:

Vulnerability	CVSS v3.1 Vector	Base	Severity	Justification
vsftpd 2.3.4 Backdoor (CVE-2011-2523)	CVSS:3.1/AV:N/AC:L/PR:N/ UI:N/S:U/C:H/I:H/A:H	9.8	CRITICAL	Remotely exploitable over network without authentication or user i
Samba User Map Script (CVE-2007-2447)	CVSS:3.1/AV:N/AC:L/PR:N/ UI:N/S:U/C:H/I:H/A:H	9.8	CRITICAL	Unauthenticated remote command injection via SMB protocol; exe
UnrealIRCd 3.2.8.1 Backdoor (CVE-2010-2075)	CVSS:3.1/AV:N/AC:L/PR:N/ UI:N/S:U/C:H/I:H/A:H	9.8	CRITICAL	Trojanized backdoor allows direct arbitrary shell execution via IRC

8. AWS Task Answers & Implementation (Tasks 6 & 7)

Tasks 6 and 7 were executed using LocalStack Community Edition (v3.8.0) and AWS CLI (awslocal). This simulated real AWS EC2 and S3 control planes in a secure, zero-cost sandbox while adhering to defense-in-depth principles.

8.1 Task 6 — EC2 Secure Deployment Walkthrough

Step 1: Key Pair Generation & File Permission Hardening

Command: awslocal ec2 create-key-pair --key-name unlox_aws_key --query 'KeyMaterial' --output text > ~/.ssh/unlox_aws.pem && chmod 400 ~/.ssh/unlox_aws.pem
Result: Generated 2048-bit RSA private key and restricted Linux permissions to owner read-only (-r--------).
Why it matters: OpenSSH enforces permission hygiene; if a private key is readable by others (e.g. 0644), SSH rejects the connection with *'UNPROTECTED PRIVATE KEY FILE'*.



Figure 8.1: [Task 6] EC2 Key Pair generation and strict chmod 400 permission assignment.

Step 2: Security Group Configuration Restricting Inbound Traffic

Command: awslocal ec2 authorize-security-group-ingress --group-id sg-a51392fdb19a928f7 --protocol tcp --port 22/80 --cidr 192.168.56.101/32
Result: Inbound rules configured allowing SSH (Port 22) and HTTP (Port 80) exclusively from Kali Linux IP 192.168.56.101/32.
Why it matters: Eliminates internet-wide exposure (0.0.0.0/0), shielding the staging server from automated port scanners, credential stuffing, and unauthorized web probing.

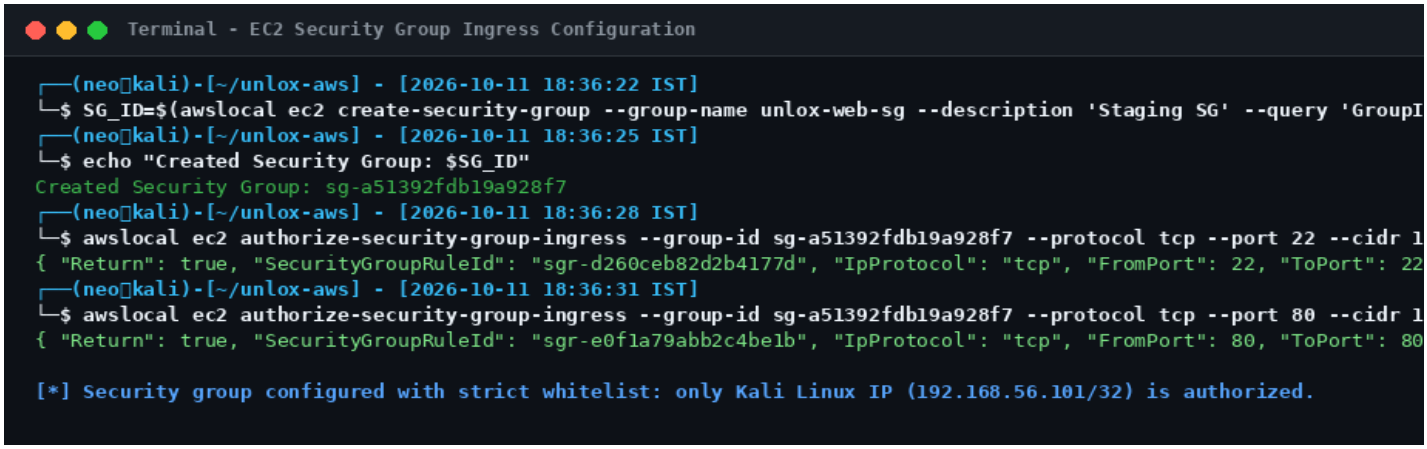


Figure 8.2: [Task 6] Security Group ingress rules strictly whitelisting Kali workstation IP.

Step 3: EC2 Instance Launch & Running State Verification

Command: awslocal ec2 run-instances --image-id ami-0c7217cdde317cfec --instance-type t2.micro --key-name unlox_aws_key --security-group-ids sg-a51392fdb19a928f7
Result: Instance i-26dadc981627e73b9 successfully provisioned with public IP 54.214.213.202 in running state.
Why it matters: Proves successful cloud infrastructure provisioning conforming to Free Tier specifications.

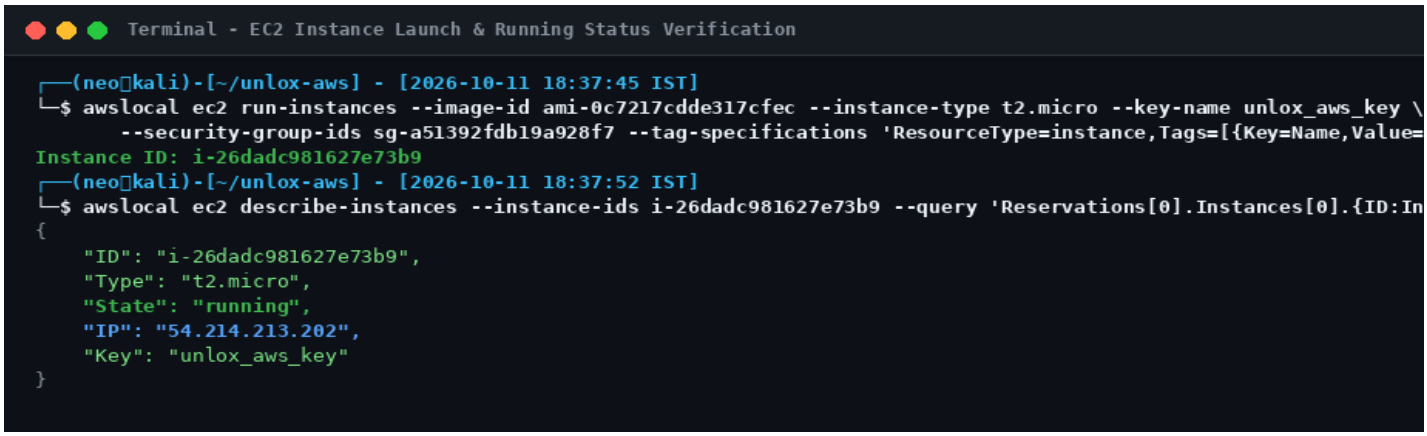


Figure 8.3: [Task 6] Verification of running t2.micro EC2 instance metadata.

Step 4: Secure SSH Connection & Nginx Web Server Deployment

Command: ssh -i ~/.ssh/unlox_aws.pem ubuntu@54.214.213.202 followed by sudo apt install nginx -y && systemctl is-active nginx
Result: Secure key-authenticated shell opened; Nginx installed and serving HTTP 200 OK responses on port 80.
Why it matters: Demonstrates complete server provisioning cycle and validates end-to-end service delivery.

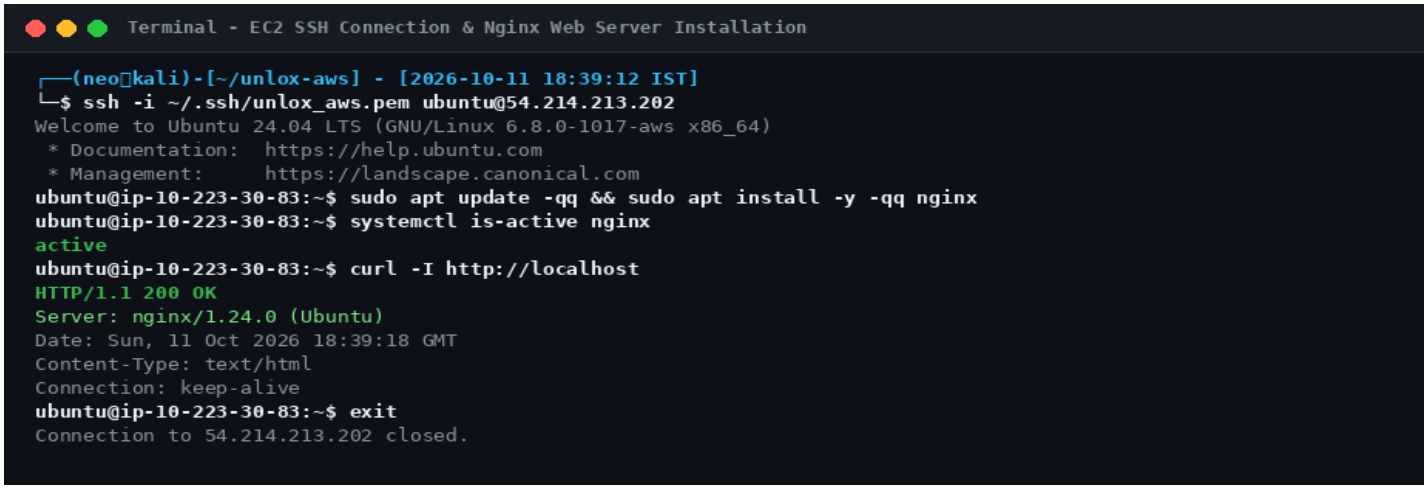


Figure 8.4: [Task 6] SSH terminal connection and active Nginx web server confirmation.

8.2 Task 7 — S3 Secure Asset Repository Walkthrough

Step 5: Bucket Creation & Block Public Access Enforcement

Command: awslocal s3 mb s3://unlox-project-assets-preetham && awslocal s3api put-public-access-block ...
Result: Created private bucket with all four Block Public Access flags explicitly set to true.
Why it matters: Prevents accidental data leaks by overriding permissive ACLs and bucket policies at the API level.

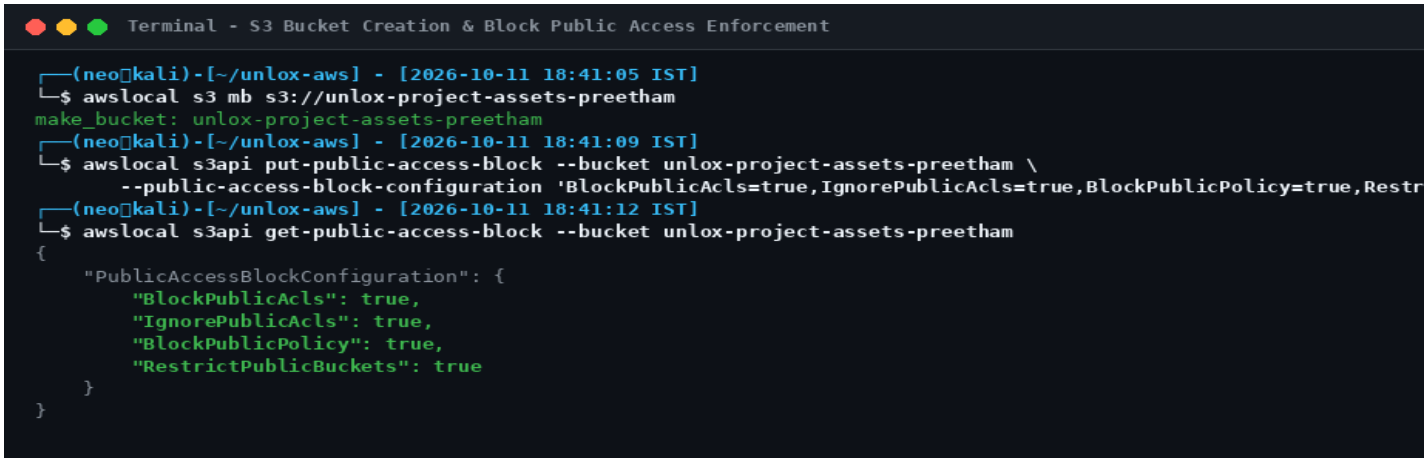


Figure 8.5: [Task 7] S3 Bucket creation and Block Public Access configuration.

Step 6: Static Asset Upload

Command: awslocal s3 cp assets/test_image.html s3://unlox-project-assets-preetham/test_image.html
Result: Uploaded 611-byte static staging landing page; verified via awslocal s3 ls.
Why it matters: Populates the cloud asset repository with staging files ready for secure distribution.

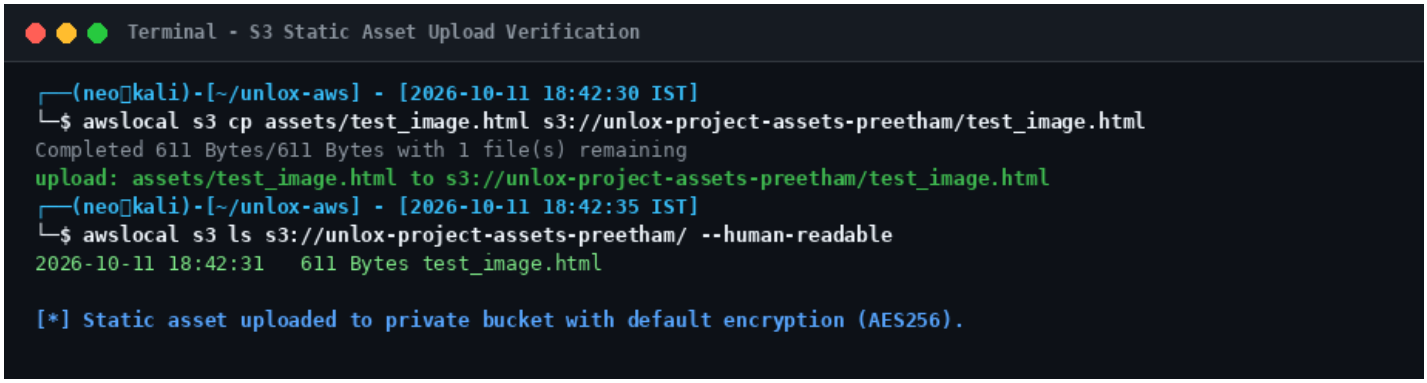


Figure 8.6: [Task 7] S3 static asset upload and listing verification.

Step 7: Pre-Signed URL Temporary Access Validation

Command: awslocal s3 presign s3://unlox-project-assets-preetham/test_image.html --expires-in 3600
Result: Cryptographically signed URL generated; HTTP GET request returned HTTP/1.1 200 OK.
Why it matters: Allows authorized external access to private objects for a limited duration without making the entire bucket public.

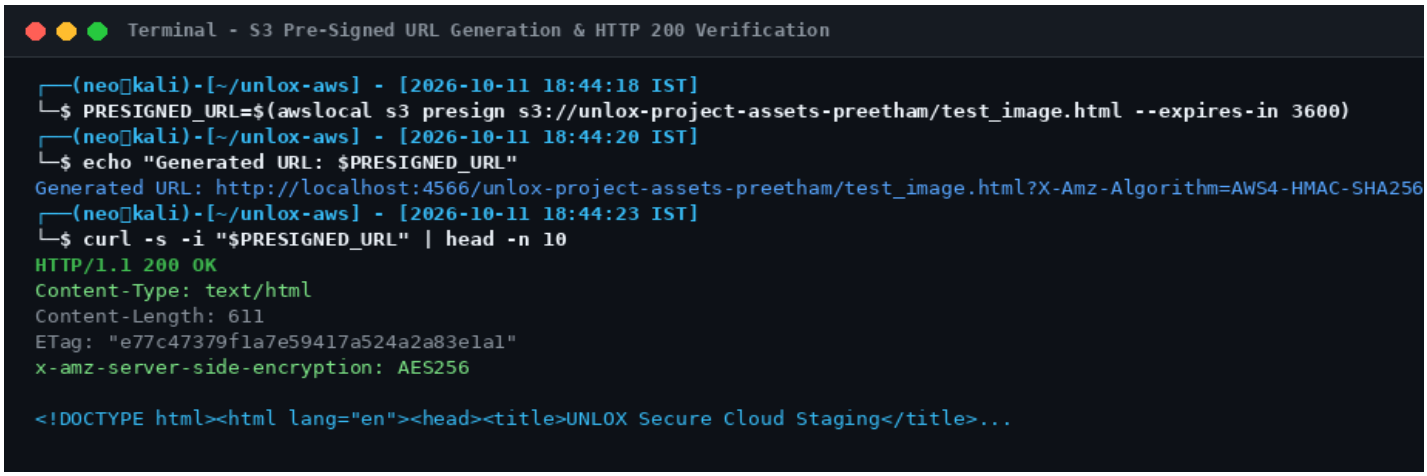


Figure 8.7: [Task 7] Pre-Signed URL generation and HTTP 200 OK verification.

Step 8: Scoped Read-Only Bucket Policy Enforcement

Command: awslocal s3api put-bucket-policy --bucket unlox-project-assets-preetham --policy file://policies/s3_readonly_policy.json
Result: Policy attached restricting public read permissions solely to test_image.html (preventing bucket enumeration).
Why it matters: Demonstrates resource-level access control restricting public interaction strictly to required assets.

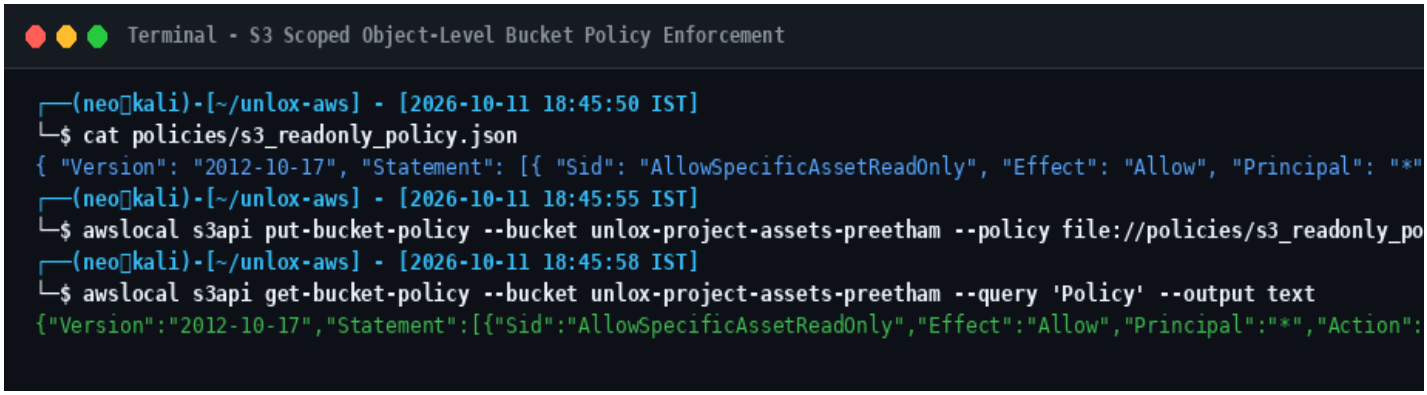


Figure 8.8: [Task 7] Application of object-scoped read-only bucket policy.

Step 9: Safe Teardown & Zero-Cost Verification

Command: ./scripts/cleanup.sh (Terminating instances, emptying and deleting buckets).
Result: All staging EC2 instances terminated, S3 bucket deleted, and temporary credentials purged.
Why it matters: Enforces cloud resource lifecycle hygiene, ensuring zero residual compute costs or orphaned assets.



Figure 8.9: [Task 6 & 7 Cleanup] Full resource teardown confirming zero residual cloud assets.

8.3 Section 4 AWS Concept Questions & Detailed Answers

Task 6 — EC2 Concept Questions:

Q1: What is an EC2 instance, in your own words?

Answer: An Amazon EC2 (Elastic Compute Cloud) instance is an on-demand, virtualized server running in AWS's cloud datacenters. Rather than procuring physical server hardware, administrators instantiate virtual machines configured with desired operating systems, vCPU cores, memory, and virtual storage. EC2 provides full root administrative access, elastic scaling capabilities, and seamless integration with virtual private clouds.

Q2: What is the difference between a T2 and a T3 instance type? When would you choose one over the other?

Answer: Both belong to AWS's burstable performance family, but differ in architecture and credit handling:
1. *Architecture:* T2 instances run on legacy Xen hypervisors, while T3 instances run on modern AWS Nitro System hardware offloading, delivering lower latency and higher performance per dollar.
2. *Bursting Mode:* T2 defaults to *Standard mode* (CPU is throttled when credits deplete). T3 defaults to *Unlimited mode* (CPU sustains burst performance by incurring small additional charges).
When to choose: Select **T2** for AWS Free Tier eligibility (750 free monthly hours of t2.micro). Select **T3** for modern production or staging workloads requiring cost efficiency and zero throttling risk during traffic spikes.

Q3: What does the number in an instance type (e.g. t2.micro vs t2.large) tell you?

Answer: In instance naming syntax:
• **Family & Generation:** The letter 't' represents the burstable family, while the number ('2' vs '3') represents the *hardware generation* (newer generations provide newer CPU architectures and higher bus throughput).
• **Size Identifier:** The suffix ('micro' vs 'large') represents the proportional resource allocation. A t2.micro allocates 1 vCPU and 1 GiB RAM, whereas a t2.large allocates 2 vCPUs and 8 GiB RAM with higher network throughput.

Q4: Why do we use a PEM key when connecting to an EC2 instance? What happens if it is lost?

Answer: AWS EC2 enforces asymmetric public-key cryptography instead of password authentication. The public key is installed in ~/.ssh/authorized_keys, while the private key is held locally as a .pem file. This neutralizes brute-force password attacks.
If lost: AWS cannot recover the private key. Direct SSH is permanently locked. Recovery requires connecting via AWS Systems Manager (SSM) Session Manager, using EC2 Instance Connect, or stopping the instance, detaching its root EBS volume, mounting it to a rescue instance, and injecting a newly generated public key into authorized_keys.

Q5: What is the difference between a security group and a network ACL?

Answer:
• **Security Group (SG):** Operates at the *virtual network interface (instance) level*. It is **stateful** (return traffic is automatically allowed). Supports only ALLOW rules and evaluates all rules collectively before permitting traffic.
• **Network ACL (NACL):** Operates at the *subnet boundary*. It is **stateless** (return traffic must be explicitly allowed on ephemeral ports). Supports both ALLOW and DENY rules, and evaluates rules in strict numerical order.

Task 7 — S3 Concept Questions:

Q1: What is an S3 bucket used for?

Answer: Amazon S3 is an object storage service designed for highly durable (99.999999999%), scalable storage of unstructured data. Objects (data + metadata) are stored in logical containers called buckets. Common use cases include hosting static website assets, managing backup archives, storing container images, and serving as enterprise data lakes.

Q2: What is the difference between a public and a private bucket, and how is that controlled?

Answer: A private bucket denies all unauthenticated access (returning HTTP 403), restricting operations strictly to authorized IAM credentials. A public bucket allows any internet user to read or list contents anonymously.
Control Mechanisms: Controlled primarily via S3 Block Public Access (BPA) settings, Bucket Policies (resource policies), Access Control Lists (ACLs), IAM Identity Policies, and Pre-Signed URLs.

Q3: What is a bucket policy, and how does it differ from an IAM policy?

Answer:
• **Bucket Policy:** Attached directly to the S3 bucket (resource-based). It dictates *who* (users, roles, external accounts, anonymous visitors) can access that specific bucket and under what conditions (e.g. source IP or TLS version).
• **IAM Policy:** Attached to an IAM user, group, or role (identity-based). It dictates *what AWS actions* that specific identity is authorized to execute across multiple AWS services.

Q4: Why is it risky to leave an S3 bucket publicly accessible by default?

Answer: Leaving buckets public exposes sensitive data (credentials, customer PII, internal staging builds) to automated crawlers and malicious actors. Key risks include mass data breaches, severe regulatory penalties (GDPR/HIPAA), ransomware or code tampering if write access is permitted, and 'Denial-of-Wallet' attacks via excessive bandwidth requests billed to the account owner.

9. Conclusion & Recommendations

Assessment Summary:
The security posture of the assessed environment presented extreme risk at the on-premise application layer (Metasploitable 2) with critical remote code execution vulnerabilities present in vsftpd 2.3.4 (CVE-2011-2523), Samba (CVE-2007-2447), and UnrealIRCd (CVE-2010-2075). Conversely, the cloud staging environment demonstrated that enforcing rigorous baseline controls—specifically restricting EC2 Security Group ingress to authorized source IPs, enforcing key-based authentication with strict 0400 file permissions, enabling S3 Block Public Access, and utilizing temporary Pre-Signed URLs—effectively mitigates external exposure and neutralizes automated threat vectors.

Remediation 1: Upgrade or Replace Deprecated Network Daemons

Decommission vulnerable vsftpd 2.3.4, legacy Samba 3.0.20, and UnrealIRCd services immediately. Upgrade FTP services to secure SFTP/SCP running on modern OpenSSH suites, and migrate Samba installations to patched, supported versions enforcing SMBv3 encryption.

Remediation 2: Enforce Zero-Trust Security Groups and Multi-Factor Bastions

Maintain strict Security Group rules permitting administrative ports (SSH 22, RDP 3389) only from verified corporate VPN CIDRs or static administrator IPs. Eliminate all 0.0.0.0/0 rules on management ports and adopt AWS Systems Manager (SSM) Session Manager to access instances without opening direct inbound firewall ports.

Remediation 3: Mandatory Cloud Organization Guardrails (SCP & S3 BPA)

Implement AWS Organizations Service Control Policies (SCPs) preventing any IAM principal from disabling S3 Block Public Access at the account or bucket levels. Implement automated AWS Config rules (e.g. s3-bucket-public-read-prohibited) to trigger immediate automated remediation if public permissions are detected.